ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №8

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Спеціальні методи (__str__, __repr__, __eq__ тощо)»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Додайте до Service та Appointment методи __str__ і __repr__, а запис порівнюйте за внутрішнім тестовим номером. Не використовуйте дані реальних осіб.

КОД РОЗВ'ЯЗКУ

```
from abc import ABC, abstractmethod

class AuditableMixin:
    """Mixin-клас, що додає функціонал логування (аудиту) подій для запису."""

    def audit_record(self):
        """Повертає рядок для запису в журнал подій медичного центру."""
        return f"[АУДИТ] Запис №{self.number} (час: {self.time}): змінено стан або створено подію."

class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def __str__(self):
        """Зручне текстове подання для користувача."""
        return f"Послуга «{self.name}» ({self.duration_min} хв)"

    def __repr__(self):
        """Технічне подання для налагодження."""
        return f"Service(name={self.name!r}, duration_min={self.duration_min!r})"

    def __eq__(self, other):
        """Порівняння послуг за назвою та тривалістю."""
        if not isinstance(other, Service):
            return NotImplemented
        return self.name == other.name and self.duration_min == other.duration_min

class Client:
    """Модель пацієнта (клієнта) медичного центру."""
```

```
        def __init__(self, full_name, phone):
            self.full_name = full_name
            self.phone = phone

        def __str__(self):
            return f"{self.full_name} (тел: {self.phone})"

        def __repr__(self):
            return f"Client(full_name={self.full_name!r},
phone={self.phone!r})"

    class Appointment(AuditableMixin, ABC):
        """Абстрактний базовий клас запису на прийом із підтримкою
магічних методів."""

        valid_statuses = {"підтверджено", "завершено", "скасовано"}
        created_count = 0

        def __init__(self, number, time, service, client,
status="підтверджено"):
            self.number = number
            self.time = time
            self.service = service
            self.client = client

            self._status = "підтверджено"
            self.status = status

            Appointment.created_count += 1

        @property
        def status(self):
            """Повертає поточний статус із захищеного внутрішнього
поля."""
            return self._status

        @status.setter
        def status(self, value):
            """Дозволяє встановити лише допустимий статус."""
            if value not in self.valid_statuses:
                raise ValueError(f"Невідомий статус: {value}")
            self._status = value

        @abstractmethod
        def preparation_note(self):
            """Абстрактний метод підготовки до прийому."""
            pass

        def __str__(self):
```

```
            """Користувацьке текстове подання запису."""
            return (
                f"Запис №{self.number} | Час: {self.time} | "
                f"Пацієнт: {self.client.full_name} | Послуга:
{self.service.name} | Статус: {self.status}"
            )

        def __repr__(self):
            """Технічне подання об'єкта для розробника."""
            return (
                f"{self.__class__.__name__}(number={self.number!r},
time={self.time!r}, "
                f"service={self.service!r}, client={self.client!r},
status={self.status!r})"
            )

        def __eq__(self, other):
            """Порівняння записів за їхнім внутрішнім номером."""
            if not isinstance(other, Appointment):
                return NotImplemented
            return self.number == other.number

    class Consultation(Appointment):
        """Похідний клас для консультаційних прийомів лікаря."""

        def __init__(self, number, time, service, client,
doctor_office, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.doctor_office = doctor_office

        def preparation_note(self):
            return f"Консультація у каб. №{self.doctor_office}:
очікуйте під кабінетом, прийдіть за 10 хв."

    class DiagnosticVisit(Appointment):
        """Похідний клас для діагностичних процедур."""

        def __init__(self, number, time, service, client,
equipment_code, status="підтверджено"):
            super().__init__(number, time, service, client, status)
            self.equipment_code = equipment_code

        def preparation_note(self):
            return f"Діагностика (обладнання код:
{self.equipment_code}): дотримуйтеся інструкцій лікаря."

    # Демонстрація роботи спеціальних методів
    if __name__ == "__main__":
        # Створення тестових об'єктів
```

```
        s1 = Service("Консультація терапевта", 30)
        s2 = Service("УЗД черевної порожнини", 40)

        c1 = Client("Олександр Т.", "+380501112233")
        c2 = Client("Марія Д.", "+380672223344")

        app1 = Consultation(1, "2026-11-05 09:00", s1, c1, "204",
"підтверджено")
        # app2 має той самий номер (1), але інші деталі для
демонстрації __eq__
        app2 = Consultation(1, "2026-11-05 14:00", s2, c2, "305",
"завершено")
        app3 = DiagnosticVisit(2, "2026-11-05 10:30", s2, c2, "EQ-
9988", "підтверджено")

        print("=== 1. Демонстрація __str__() та __repr__() ===")
        print("Виклик print(app1) -> використовує __str__:")
        print(app1)

        print("\nВиклик repr(app1) -> використовує __repr__:")
        print(repr(app1))

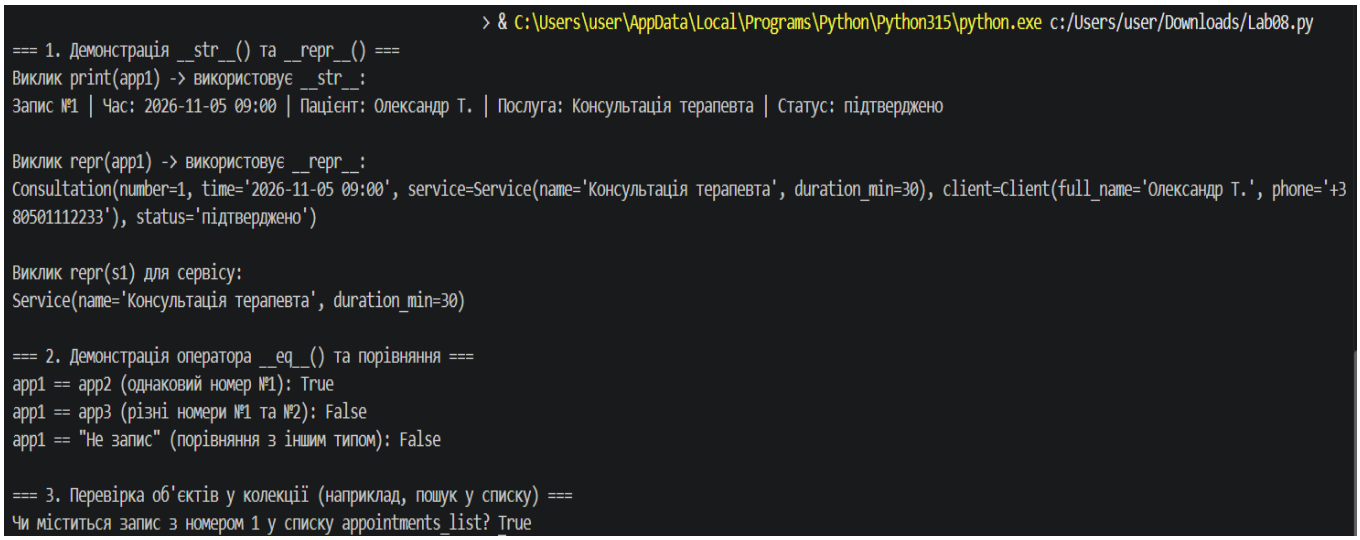
        print("\nВиклик repr(s1) для сервісу:")
        print(repr(s1))

        print("\n=== 2. Демонстрація оператора __eq__() та
порівняння ===")
        print(f"app1 == app2 (однаковий номер №1): {app1 == app2}")
        print(f"app1 == app3 (різні номери №1 та №2): {app1 ==
app3}")
        print(f"app1 == \"Не запис\" (порівняння з іншим типом):
{app1 == 'Не запис'}")

        print("\n=== 3. Перевірка об'єктів у колекції (наприклад,
пошук у списку) ===")
        appointments_list = [app1, app3]

        # Шукаємо екземпляр за допомогою оператора `in`, який
використовує __eq__
        search_target = Consultation(1, "2026-11-05 00:00", s1, c1,
"скасовано")
        print(f"Чи міститься запис з номером 1 у списку
appointments_list? {search_target in appointments_list}")
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ



```
                                                > & C:\Users\user\AppData\Local\Programs\Python\Python315\python.exe c:/Users/user/Downloads/Lab08.py
=== 1. Демонстрація __str__() та __repr__() ===
Виклик print(app1) -> використовує __str__:
Запис №1 | Час: 2026-11-05 09:00 | Пацієнт: Олександр Т. | Послуга: Консультація терапевта | Статус: підтверджено

Виклик repr(app1) -> використовує __repr__:
Consultation(number=1, time='2026-11-05 09:00', service=Service(name='Консультація терапевта', duration_min=30), client=Client(full_name='Олександр Т.', phone='+3
80501112233'), status='підтверджено')

Виклик repr(s1) для сервісу:
Service(name='Консультація терапевта', duration_min=30)

=== 2. Демонстрація оператора __eq__() та порівняння ===
app1 == app2 (однаковий номер №1): True
app1 == app3 (різні номери №1 та №2): False
app1 == "Не запис" (порівняння з іншим типом): False

=== 3. Перевірка об'єктів у колекції (наприклад, пошук у списку) ===
Чи міститься запис з номером 1 у списку appointments_list? True
```